



Website Delivery with CloudFront



Seye Sanya

User Information





Introducing Today's Project!

In this project, I will demonstrate how to use CloudFront to deliver a website globally. I'm doing this project to learn content delivery networks (CDNs) and to set up the presentation tier of a three-tier architecture.

Tools and concepts

Services I used were Amazon S3 and CloudFront. Key concepts I learnt include content delivery network (CDN), distribution, origin access control, performance load times and S3 static website hosting vs CloudFront.

Project reflection

This project took me approximately three hours, including demo time. The most challenging part was having a grasp of the idea behind Origin Access Control (OAC). It was most rewarding to compare performance lead times and see CloudFront outperforming S3.

I chose to do this project today because to learn about CloudFront and Content Delivery. The project met my goals and I successfully built the presentation tier of a three-tier architecture. Something that would make learning with NextWork even better is...



Set Up S3 and Website Files

I started the project by creating an S3 bucket to store my website's files. I can't use CloudFront for this task because it is not a storage service but a delivery service for objects already stored elsewhere.

The three files that make up my website are index.html, which outlines the text and images of my website, style.css, which helps structure the elements properly and script.js, which give the design layout better appeal and adds fun colors and elements

I validated that my website files work by opening all three files on my mac. They all worked perfectly.

Upload succeeded
For more information, see the Files and folders table.

Destination
s3://nextwork-three-tier-sheye-123

Succeeded
3 files, 1.7 KB (100.00%)

Failed
0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (3 total, 1.7 KB)

Find by name

Name	Folder	Type	Size	Status	Error
script.js	-	text/javascript	680.0 B	Succeeded	-
style.css	-	text/css	447.0 B	Succeeded	-
index (1).html	-	text/html	564.0 B	Succeeded	-



Exploring Amazon CloudFront

Amazon CloudFront is a Content Delivery Network (CDN), which means it speeds up the distribution of your static and dynamic web content, such as .html, .css, .js, and image files. Businesses and developers use CloudFront because it speeds up a website's performance

To use Amazon CloudFront, you set up distributions, which are instructions that tell cloudfront how to deliver your content. I set up a distribution for our website. The origin is my s3 bucket.

My CloudFront distribution's default root object is index(1).html. This means when users visit the url, they will be able to see the contents of index(1).html.

Origin

S3 origin

Choose an AWS origin, or enter your origin's domain name. [Learn more](#)

nextwork-three-tier-sheye-123.s3.us-east-2.amazonaws.com

Origin path - *optional*

The directory path within your origin where your content is stored. [Learn more](#)

/path



Handling Access Issues

When I tried visiting my distributed website, I ran into an access denied error because the file in question does not appear to have any style information associated with it.

My distribution's origin access settings was public. This caused the access denied error because the s3 bucket should have also been public but it was not.

To resolve the error, I set up origin access control (OAC). OAC is a special control system that lets you keep your S3 bucket and objects not publicly accessible, while still making sure they can be accessed through CloudFront.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

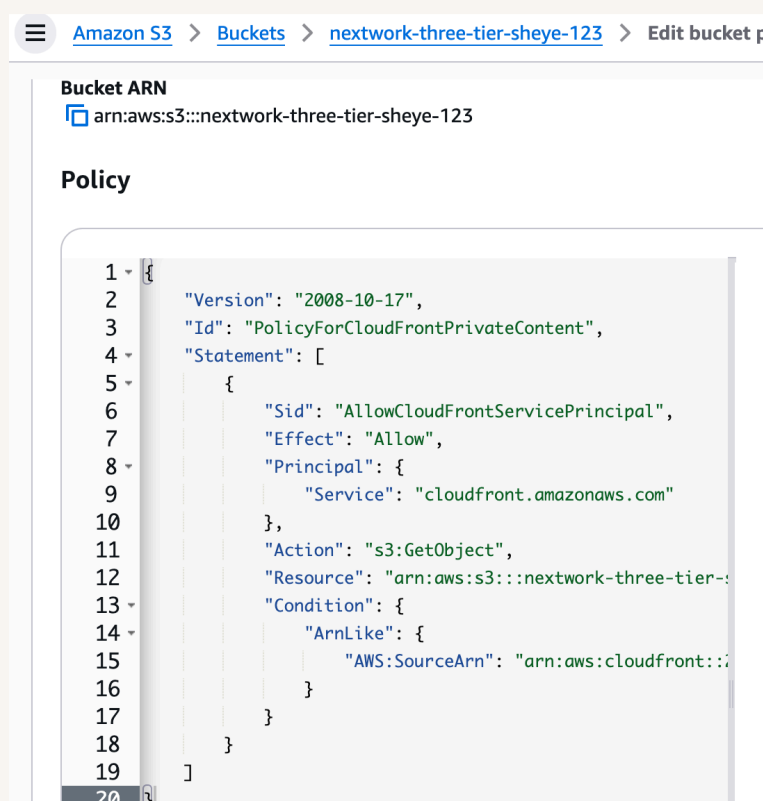
```
▼<Error>
  <Code>AccessDenied</Code>
  <Message>Access Denied</Message>
</Error>
```



Updating S3 Permissions

Once I set up my OAC, I still needed to update my bucket policy because it's objects are still private to everyone including CloudFront. Just creating the OAC does not mean the CloudFront automatically connected to the s3.

Creating an OAC automatically gives me a policy I could copy, which grants the CloudFront service permission to read objects from my S3 bucket on behalf of my CloudFront distribution. This allows CloudFront to securely retrieve and serve the files in the private S3 bucket without making the bucket publicly accessible.



```
1 {
2   "Version": "2008-10-17",
3   "Id": "PolicyForCloudFrontPrivateContent",
4   "Statement": [
5     {
6       "Sid": "AllowCloudFrontServicePrincipal",
7       "Effect": "Allow",
8       "Principal": {
9         "Service": "cloudfront.amazonaws.com"
10      },
11      "Action": "s3:GetObject",
12      "Resource": "arn:aws:s3:::nextwork-three-tier-sheye-123",
13      "Condition": {
14        "ArnLike": {
15          "AWS:SourceArn": "arn:aws:cloudfront::*"
16        }
17      }
18    ]
19  }
```



S3 vs CloudFront for Hosting

For my project extension, I'm comparing S3 with CloudFront and I initially had an error with static website hosting because we enabled s3 static website hosting without making the object available.

I tried resolving this by turning off block public access in my s3 bucket. I still ran into an error because the bucket policy still does not allow public access to the bucket yet.

I could finally see my S3 hosted website when I updated my bucket policy to allow the bucket policy have access to objects. This worked because the previous policy only allowed CloudFront access, as changing s3 permissions is required.

Compared to the permission settings for my CloudFront distribution, using S3 meant I had to enable public access to my bucket. I preferred using CloudFront because it meant bucket access is restricted to CloudFront only which is great for security.

S3 vs CloudFront Load Times

Load time means how long it takes for my browser to receive the exploited content and display it to the user. The load times for the CloudFront site were faster than the S3 site because content is cache from edge locations.

A business would prefer CloudFront when performance loadtimes are most important. S3 static website hosting might be sufficient when one is hosting a simple website without loadtime requirements.

User Information

Get User Data

☐

dbchzf1xri3pv.cloudfront.net

1 error, 6 warnings, 1 info

Filter

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

WS

Wasm

Other

100 ms

200 ms

300 ms

400 ms

500 ms

600 ms

Name	Status	Type	Initiator	Size	Time
dbchzf1xri3pv.cloudfront.net	304	docum...	Other	272 B	63 ms
style.css	304	stylesh...	(index):6	272 B	59 ms
script.js	304	script	(index):15	271 B	64 ms
favicon.ico	403	xml	Other	351 B	307 ms

4 requests

1.2 kB transferred

1.8 kB resources

Finish: 606 ms

DOMContentLoaded: 2



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

